
Daily Progress Report

Week 6_Day3: Performance Hooks & Refs

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_6 (Day 3)
Date	29 July, 2026

Day 3: Performance Hooks & Refs

1.1 Topics Studied

- Understanding React performance optimization techniques
- Preventing unnecessary re-renders using `useMemo`
- Memoizing callback functions using `useCallback`
- Understanding when memoization is beneficial in React applications
- Accessing and manipulating DOM elements directly using `useRef`
- Working with the HTML5 `<video>` element through React refs
- Implementing custom video controls without relying on default browser controls
- Managing video playback state (Play/Pause)
- Controlling video audio state (Mute/Unmute)
- Tracking video progress and playback time
- Implementing custom seek functionality using a progress slider
- Formatting video duration and current playback time
- Building reusable and maintainable React components
- Designing responsive user interfaces with modern CSS

1.2 Task Description

Developed a professional **Custom Video Player** in React to demonstrate the practical implementation of **`useRef`**, **`useMemo`**, and **`useCallback`**. The application replaced the default browser video controls with a fully customized control panel, allowing users to play, pause, mute, unmute, and seek through the video using custom React components.

The HTML5 <video> element was accessed directly through the **useRef** hook, enabling complete programmatic control over video playback. Performance optimization techniques were incorporated by memoizing frequently used callback functions with **useCallback**, preventing unnecessary function recreation during component re-renders. Additionally, formatted video duration and current playback time were optimized using **useMemo**, ensuring computations occurred only when their dependencies changed.

The application continuously tracked the video's playback progress, updated the progress bar in real time, and displayed the current playback position alongside the total video duration. A responsive and modern user interface was implemented using custom CSS, providing an improved viewing experience across desktop and mobile devices while maintaining a clean and professional design.

1.3 Implementation Details

- Developed a reusable VideoPlayer component.
- Used the useRef() hook to obtain direct access to the HTML5 <video> element.
- Implemented custom Play and Pause functionality without browser controls.
- Added custom Mute and Unmute functionality using direct DOM manipulation through the video reference.
- Disabled the default HTML5 video controls and replaced them with fully customized React controls.
- Managed video playback state using the useState() hook.
- Optimized event handler functions using useCallback() to reduce unnecessary function recreation during re-renders.
- Optimized formatted playback time and total duration using useMemo().
- Implemented a custom progress bar that updates continuously during video playback.
- Enabled users to seek to any position in the video using the custom slider.
- Displayed both current playback time and total video duration in MM format.
- Displayed a loading indicator until the video's metadata was fully loaded.
- Designed a responsive and modern user interface using reusable CSS components.
- Organized the project into reusable component and styling files to improve maintainability and scalability.

1.4 Outcome

Successfully developed a professional **Custom Video Player** that demonstrates the practical implementation of React Performance Hooks and Refs. The project strengthened the understanding of accessing DOM elements using **useRef**, optimizing React applications through **useMemo** and **useCallback**, and creating fully customized media controls without relying on browser-provided functionality. Furthermore, implementing responsive layouts, reusable components, and optimized rendering techniques reinforced best practices for building scalable, maintainable, and high-performance React applications.

Screenshots:

```
6 function VideoPlayer() {
16   const togglePlay = useCallback(() => {
20
21     if (player.paused) {
22       player.play();
23       setIsPlaying(true);
24     } else {
25       player.pause();
26       setIsPlaying(false);
27     }
28   }, []);
29
30   const toggleMute = useCallback(() => {
31     const player = videoRef.current;
32
33     if (!player) return;
34
35     const newMutedState = !player.muted;
36     player.muted = newMutedState;
37     setMuted(newMutedState);
38   }, []);
39
40   const updateProgress = useCallback(() => {
41     const player = videoRef.current;
42
43     if (!player) return;
44
45     const value = (player.currentTime / player.duration) * 100;
46
```

```
63   useEffect(() => {
65
66     const loaded = () => {
67       setDuration(player.duration);
68       setLoading(false);
69     };
70
71     player.addEventListener("loadedmetadata", loaded);
72     player.addEventListener("timeupdate", updateProgress);
73
74     return () => {
75       player.removeEventListener("loadedmetadata", loaded);
76       player.removeEventListener("timeupdate", updateProgress);
77     };
78   }, [updateProgress]);
79
80   const formatTime = useCallback((time) => {
81     const minutes = Math.floor(time / 60);
82
83     const seconds = Math.floor(time % 60);
84
85     return `${minutes}:${seconds.toString().padStart(2, "0")}`;
86   }, []);
87
88   const totalDuration = useMemo(() => {
89     return formatTime(duration);
90   }, [duration, formatTime]);
91
```

Screenshots:

Custom React Video Player

Built using `useRef`, `useMemo` and `useCallback`.

